

Building a Distributed Stream Processing Engine

Project companion — phased implementation guide (Java)

This companion accompanies **Part 9A: Stream Processing Fundamentals** of the *Developing Intuition on Building Blocks - Systems Design* series. The article explains how a distributed stream processing engine works. This document is the build guide for a working one.

What this document is

A phased implementation guide with the core interfaces, wire protocol and critical algorithm snippets spelled out, so the engine can be built incrementally - each phase runnable and demonstrable before the next begins. It is not a full source listing; the complete code lives in the GitHub repository alongside a `CLAUDE.md` handoff file for driving the build with a code-generation tool.

Contents

1. Scope: A Teaching Engine, Honestly Bounded
2. The Business Use Case
3. Requirements: Functional and Non-Functional
4. Architecture
5. Core Abstractions
6. The Control Plane: gRPC Contracts
7. Phase 1 - Single-Process Engine
8. Phase 2 - Master and Worker Processes
9. Phase 3 - Event Time, Watermarks and Session Windows
10. Phase 4 - Checkpointing, Barrier Alignment and Recovery
11. Phase 5 - The Interval Join
12. Phase 6 - The Transactional Iceberg Sink
13. Phase 7 - Status API and the Four Demos
14. Local Environment
15. Known Limitations and Extensions

1. Scope: A Teaching Engine, Honestly Bounded

The goal is not to compete with Apache Flink. The goal is to be forced into making, explicitly and in code, the same design decisions Flink's authors made - and to feel why each one exists. When Part 9B walks through Flink's JobManager or Storm's Nimbus, you will be reading about a component whose job you have already done yourself.

What the engine does

- Accepts a user-defined job as a DAG of operators and compiles it to a physical execution plan.
- Distributes tasks across genuinely separate worker processes, communicating over gRPC.
- Tracks event time with watermarks; evaluates keyed, event-time session windows.
- Takes consistent distributed checkpoints using barrier alignment, and recovers from them.
- Commits results transactionally to an Apache Iceberg table, exactly once.
- Exposes job, task and checkpoint state through a status API.

What it deliberately does not do

- No SQL layer, no query optimizer, no cost-based planning.
- No dynamic rescaling of a running job. Parallelism is fixed at submission.
- No standby master with leader election. The master is a single point of failure by default - exactly as Flink's JobManager is without HA configured (Article Section 4.5). Metadata lives in etcd so recovery is possible; automatic failover is listed as an extension.
- No security, multi-tenancy, resource isolation or backpressure-aware autoscaling.
- Unaligned checkpoints are not implemented. Alignment is the mechanism being taught.

Stated limitations are a feature, not an apology

Every shortcut below is named in the article and in this document rather than quietly implied away. Knowing precisely where a teaching implementation stops is part of understanding what production engines are actually doing for you.

2. The Business Use Case

The LMS (Library Management System) carries a catalog-browse clickstream: members searching the catalog, clicking search results, and sometimes borrowing what they clicked. Today that behaviour is only queryable through the warehouse built in Part 7A, which means every question is a question about yesterday.

The collections team wants three answers:

- **Which search terms produce clicks but no borrows** - a signal that catalog metadata is misleading, or the item was unavailable by the time the member reached it.
- **How long members browse before borrowing** - session-shaped behaviour, not row-shaped.
- **Which newly-added titles are getting attention right now** - a question whose value evaporates entirely if the answer is a day old.

Why the existing systems in this series do not answer it

System	Covered in	Why it falls short here
Relational database	Parts 2, 4a	Can store the events, but "sessions per member with a 15-minute inactivity gap" means re-scanning and re-grouping the full day on every query run.
Warehouse / batch ETL	Part 7A	Correct and efficient, once per scheduled run. The freshness gap is structural, not a tuning problem.
MapReduce / Spark batch	Parts 7B, 7C	Recomputes from scratch each run. Running it hourly instead of daily buys less staleness at the cost of re-scanning a growing dataset every hour.
Stream processing engine	This part	Each new click incrementally updates an answer that already exists. State and time are managed by the engine rather than worked around by the reporting layer.

The event model

Two input streams, both keyed by member, both carrying an event-time timestamp assigned at the client:

```
// Topic: lms.catalog.clicks
record ClickEvent(
    String memberId,          // partition + state key
    String catalogItemId,
    String searchTerm,
    String eventType,         // SEARCH | RESULT_CLICK
    long   eventTimeMillis    // assigned at the client, not on arrival
) {}

// Topic: lms.catalog.borrows
record BorrowEvent(
    String memberId,
    String catalogItemId,
    String loanId,
    long   eventTimeMillis
) {}
```

Both topics are partitioned on memberId so per-member ordering holds within a partition (Part 3, key-based partitioning).

3. Requirements: Functional and Non-Functional

3.1 Functional

The engine must run a job composed of four operator types, chosen because together they exercise every core mechanism the article covers:

Operator	What it does in the LMS job	Mechanism exercised (article section)
Stateless filter	Drops malformed and bot-flagged click events.	Task execution, forward exchange (S3.4), task chaining (S3.5)
Keyed session aggregation	Per member: click count, distinct search terms, session duration, closed on a 15-minute inactivity gap.	Keyed state (S9.1), event-time session windows (S7.1), watermarks (S6.2), state TTL (S9.4)
Interval join	Matches a RESULT_CLICK to a BORROW for the same member and item within 30 minutes.	Two-sided buffered state, watermark-driven eviction (S8.2-8.3)
Iceberg sink	Appends session and conversion rows to an Iceberg analytics table.	Checkpoint-coordinated transactional commit (S11.3-11.4)

3.2 Non-Functional

Three requirements shape the architecture. Each forces a specific mechanism into the design - they are not features chosen for their own sake.

Requirement	Concretely	What it forces into the design
Correctness under failure	A worker dying mid-window must not produce a double-counted or missing session row.	Consistent checkpointing with barrier alignment (Phase 4) and a transactional sink (Phase 6). Nothing weaker meets the requirement.
Freshness	Results visible within seconds of the event, not hours.	Rules out a sink that batches on a wall-clock schedule. With a transactional sink, the checkpoint interval directly sets result visibility latency - making it a product decision, not only an operational one.
Observable job state	Answerable from outside the cluster at any moment: is the job running, which tasks are alive, when did the last checkpoint complete, what is the source lag.	A status API on the master plus per-task metric reporting on the heartbeat (Phase 7).

The freshness / correctness dial

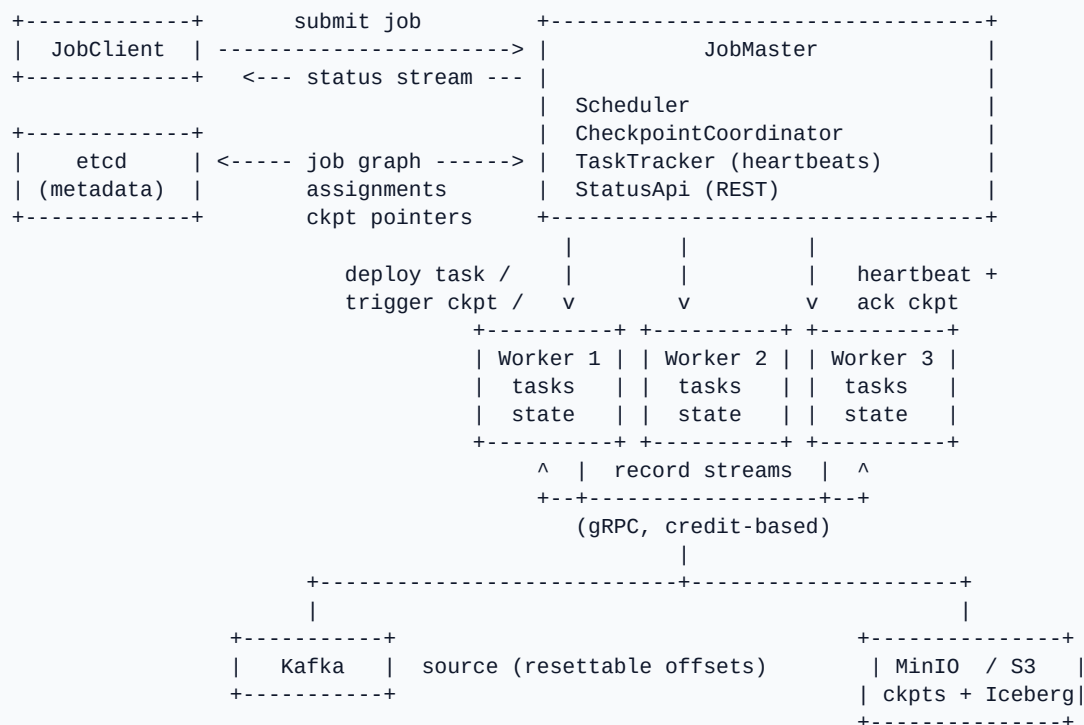
A shorter checkpoint interval means fresher visible results and less reprocessing after a failure, at the cost of more frequent checkpoint overhead and more Iceberg commits (and so more small files). Default the project to a 10-second interval and make it configurable - Demo 4 is more interesting when you can move this dial and watch what changes.

4. Architecture

The component list falls out of Section 3 almost mechanically, which is the point.

Component	Process	Responsibility
JobClient	CLI	Builds the job graph from user code, serializes it, submits to the master, streams back status.
JobMaster	Master	Compiles the logical graph to a physical execution graph; assigns tasks to workers; tracks liveness by heartbeat; owns the job state machine.
CheckpointCoordinator	Master (component)	Triggers checkpoints on an interval, instructs source tasks to inject barriers, collects per-task acknowledgements, marks a checkpoint complete, notifies sinks to commit.
MetadataStore	etcd	Durable home for the job graph, task assignments and completed-checkpoint pointers, so a restarted master recovers rather than losing the job.
TaskWorker	Worker (xN)	Registers with the master, runs assigned tasks on its own threads, owns local state, exchanges records with peer workers.
StateBackend	Worker (component)	Embedded per-task key-value store for keyed state; snapshots to shared durable storage on checkpoint.
RecordTransport	Worker (component)	Serializes records, routes them by exchange strategy, ships them over gRPC streams to peer workers, applies credit-based flow control.
StatusApi	Master (component)	REST endpoint exposing job state, per-task state, checkpoint history and source lag.

4.1 Process topology



Every arrow in this diagram maps to a mechanism in the article: *deploy/assign* (S4.3), *heartbeat* (S4.4), *barrier injection* (S10.2), *record streams* (S5.2), *credit* (S5.3).

5. Core Abstractions

Five interfaces carry the whole design. Getting these right in Phase 1 is what makes Phases 2 through 6 additive rather than rewrites.

5.1 The record envelope

Everything travelling between tasks is a `StreamElement`: either a data record, a watermark, or a checkpoint barrier. Control elements flowing in-band with data is the single most important structural decision in the engine - it is what makes both watermark propagation (S6.3) and barrier alignment (S10.3) possible.

```
public sealed interface StreamElement
    permits StreamRecord, Watermark, CheckpointBarrier {}

/** A user data record plus its event-time timestamp. */
public record StreamRecord<T>(T value, long timestamp) implements StreamElement {}

/** Asserts: no element with eventTime < timestamp will follow on this channel. */
public record Watermark(long timestamp) implements StreamElement {
    public static final Watermark MAX = new Watermark(Long.MAX_VALUE);
}

/** Logically splits the stream into pre- and post-checkpoint halves. */
public record CheckpointBarrier(long checkpointId, long triggerTimestamp)
    implements StreamElement {}
```

5.2 Operators

A user-facing functional interface for simple cases, and a richer one for operators that need state and timers. The split mirrors Flink's `MapFunction` vs `KeyedProcessFunction` distinction.

```
public interface Operator<IN, OUT> {
    /** Called once per record. Emit zero or more outputs via the collector. */
    void processElement(StreamRecord<IN> record, Collector<OUT> out) throws Exception;

    /** Called when this operator's event-time clock advances (min across inputs). */
    default void onWatermark(long watermark, Collector<OUT> out) throws Exception {}

    default void open(OperatorContext ctx) throws Exception {}
    default void close() throws Exception {}
}

public interface KeyedOperator<K, IN, OUT> extends Operator<IN, OUT> {
    /** Fired when a previously registered event-time timer's time has been reached.
     * The runtime restores the timer's key context before invoking this. */
    void onEventTimer(long timestamp, K key, Collector<OUT> out) throws Exception;
}

public interface OperatorContext {
    <T> ValueState<T> getValueState(String name, Class<T> type);
    <T> ListState<T> getListState(String name, Class<T> type);
    void registerEventTimer(long timestamp); // scoped to the current key
    long currentWatermark();
    MetricGroup metrics();
}
```

5.3 State

Keyed state only - operator state is not needed for this job beyond source offsets, which the source task handles directly. The backend interface is deliberately narrow so an in-memory implementation can be swapped for

RocksDB without touching operator code (S9.3).

```
public interface ValueState<T> {
    T value() throws IOException;           // scoped to the current key
    void update(T value) throws IOException;
    void clear();
}

public interface StateBackend extends AutoCloseable {
    /** All access below is implicitly scoped to the key set by setCurrentKey. */
    void setCurrentKey(Object key);

    <T> ValueState<T> valueState(String stateName, Class<T> type);
    <T> ListState<T> listState(String stateName, Class<T> type);

    /** Write a snapshot of all state owned by this task. Returns a handle
     *  (a URI) the master persists in the checkpoint metadata. */
    StateHandle snapshot(long checkpointId, Path checkpointDir) throws IOException;

    /** Replace all local state with the contents of a previously written handle. */
    void restore(StateHandle handle) throws IOException;
}
```

Phase 1 ships InMemoryStateBackend (HashMap per state name). Phase 4 adds RocksDbStateBackend using org.rocksdb:rocksdbjni, one column family per state name.

5.4 Job graph

The user-facing builder produces a logical graph; the master compiles it to a physical one (S3.1). Keeping these as two distinct types in code, rather than mutating one, is what keeps the distinction legible.

```
// ---- what the user writes (logical) ----
JobGraph job = JobGraph.named("lms-clickstream")
    .source("clicks", KafkaSource.of("lms.catalog.clicks", ClickEvent.class))
    .source("borrows", KafkaSource.of("lms.catalog.borrows", BorrowEvent.class))
    .withEventTime(ClickEvent::eventTimeMillis, Duration.ofSeconds(5)) // out-of-orderness
    .filter("drop-bots", new BotFilter()).parallelism(4)
    .keyBy("by-member", ClickEvent::memberId)
    .window("sessions", SessionWindows.withGap(Duration.ofMinutes(15)))
    .aggregate(new SessionAggregator()).parallelism(4)
    .intervalJoin("click-to-borrow")
    .between(Duration.ZERO, Duration.ofMinutes(30))
    .process(new ConversionJoin()).parallelism(4)
    .sink("iceberg", new IcebergSink("lms.analytics.browse_sessions")).parallelism(2)
    .build();

// ---- what the master compiles it into (physical) ----
public record ExecutionVertex(String operatorId, int subtaskIndex, int parallelism,
    ExchangeStrategy inputExchange, List<String> upstreamIds) {}
public record ExecutionGraph(String jobId, List<ExecutionVertex> vertices,
    Map<String, TaskAssignment> assignments) {}
```

operatorId is assigned explicitly by the user-facing name ("sessions", "iceberg") and never derived from graph position - this is what makes savepoint-compatible job evolution possible (Article S12.3).

5.5 Exchange strategies

```
public enum ExchangeStrategy {
    FORWARD,    // 1:1 to the co-located downstream subtask; chainable
    HASH,        // partition by key hash -> guarantees same key, same subtask
    BROADCAST,   // every record to every downstream subtask
    REBALANCE    // round-robin; load balancing only, NEVER for keyed operators
}

/** Key groups decouple key->subtask assignment from current parallelism (S9.5). */
public final class KeyGroupAssigner {
    public static final int MAX_PARALLELISM = 128;    // == number of key groups

    public static int keyGroupFor(Object key) {
        return Math.abs(murmurHash(key.hashCode())) % MAX_PARALLELISM;
    }
    public static int subtaskFor(Object key, int parallelism) {
        return keyGroupFor(key) * parallelism / MAX_PARALLELISM;
    }
}
```

MAX_PARALLELISM is fixed at job creation and cannot change later without discarding state - the same constraint real engines impose, and worth feeling directly.

6. The Control Plane: gRPC Contracts

Two services. The master drives task lifecycle and checkpoints; workers report liveness and acknowledge. A third, separate streaming service carries data records between workers - deliberately kept off the control plane so a slow data path cannot starve heartbeats.

```
service MasterService {
  rpc RegisterWorker (WorkerInfo) returns (RegisterAck);
  rpc Heartbeat (stream WorkerBeat) returns (stream MasterCommand);
  rpc AcknowledgeCheckpoint (CheckpointAck) returns (Empty);
  rpc ReportTaskFailure (TaskFailure) returns (Empty);
}

service WorkerService {
  rpc DeployTask (TaskDeployment) returns (DeployAck);
  rpc CancelTask (TaskId) returns (Empty);
  rpc TriggerCheckpoint (CheckpointTrigger) returns (Empty); // sources only
  rpc NotifyCheckpointComplete (CheckpointId) returns (Empty); // sinks commit here
  rpc RestoreTask (RestoreRequest) returns (Empty);
}

service DataTransportService {
  // One long-lived bidirectional stream per (sender subtask, receiver subtask) pair.
  // Receiver sends CreditGrant upstream; sender ships buffers within its credit.
  rpc ExchangeRecords (stream DataBuffer) returns (stream CreditGrant);
}
```

6.1 Key messages

```
message WorkerBeat {
  string worker_id = 1;
  repeated TaskStatus tasks = 2; // RUNNING | FAILED | FINISHED, per subtask
  int64 current_watermark = 3; // lowest across this worker's tasks
  int64 records_processed = 4;
  int64 source_lag_millis = 5; // for the status API
}

message CheckpointTrigger { int64 checkpoint_id = 1; int64 trigger_ts = 2; }

message CheckpointAck {
  string worker_id = 1;
  string task_id = 2;
  int64 checkpoint_id = 3;
  string state_handle_uri = 4; // where this task wrote its snapshot
  int64 alignment_millis = 5; // observability: S10.5's cost, measured
  int64 state_size_bytes = 6;
}

message DataBuffer {
  string sender_task_id = 1;
  int32 target_subtask = 2;
  bytes payload = 3; // N serialized StreamElements, batched
  int32 element_count = 4;
}
```

Batching multiple elements per DataBuffer is the project's version of Flink's network buffers (S5.2): flush when full, or when a configurable buffer timeout expires - the latency/throughput dial from the article, exposed as a config value.

7. Phase 1 - Single-Process Engine

Phase goal

A single JVM reads from Kafka, runs a chain of operators against it, and prints results. No distribution, no state, no checkpoints. Proves the abstractions in Section 5 hold together before any network is involved.

7.1 Build order

- Define the `StreamElement` hierarchy and the `Operator` interfaces exactly as in Section 5.
- Implement `JobGraph` as a fluent builder producing an immutable logical graph. Do not compile to a physical graph yet - just validate the DAG (no cycles, every operator reachable from a source, parallelism greater than zero).
- Implement a `Task` as a `Runnable` owning an input queue, one operator instance, and a `Collector` that pushes to the next task's queue. This queue is the project's stand-in for Storm's disruptor queue (S5.1).
- Implement `KafkaSourceTask` around a plain `KafkaConsumer`; track offsets in a field for now.
- Implement `BotFilter` (stateless) and a `PrintSink`. Run the two-operator job end to end.

7.2 The task run loop

This loop is the heart of the engine and barely changes across later phases - Phase 3 adds the watermark branch, Phase 4 adds the barrier branch. Writing it with that shape from the start avoids a rewrite.

```
public void run() {
    try {
        operator.open(context);
        while (running) {
            StreamElement element = inputQueue.poll(100, TimeUnit.MILLISECONDS);
            if (element == null) continue;

            switch (element) {
                case StreamRecord<?> record -> {
                    if (keyed) stateBackend.setCurrentKey(keySelector.apply(record.value()));
                    operator.processElement((StreamRecord) record, collector);
                    metrics.recordsIn.increment();
                }
                // Phase 3 fills these in:
                case Watermark w -> handleWatermark(w);
                // Phase 4 fills this in:
                case CheckpointBarrier b -> handleBarrier(b);
            }
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } catch (Exception e) {
        master.reportTaskFailure(taskId, e); // Phase 2 onward
    } finally {
        closeQuietly(operator);
    }
}
```

A single-threaded task with an inbound queue means operator code never needs to be thread-safe - the same guarantee real engines provide, and worth preserving deliberately.

7.3 Done when

- The job runs from a Kafka topic through filter to console output.

- Killing the process and restarting it reprocesses from the committed consumer offset (at-least-once, no state) - a useful baseline to contrast against Phase 4.

8. Phase 2 - Master and Worker Processes

Phase goal

Split into one master process and N worker processes. The master compiles the physical graph, assigns tasks, and tracks liveness. Workers exchange records over gRPC. Still no state, still no checkpoints.

8.1 Scheduling

Keep it deliberately simple: round-robin subtasks across registered workers, subject to one rule - all subtasks of a chained group land together. A slot in this project is one vertical pipeline slice, mirroring Flink rather than Storm (Article S4.2).

```
public ExecutionGraph compile(JobGraph logical, List<WorkerInfo> workers) {
    List<ExecutionVertex> vertices = new ArrayList<>();
    for (LogicalOperator op : logical.operatorsInTopologicalOrder()) {
        for (int i = 0; i < op.parallelism(); i++) {
            vertices.add(new ExecutionVertex(
                op.id(), i, op.parallelism(),
                exchangeFor(op), // FORWARD | HASH | BROADCAST | REBALANCE
                op.upstreamIds()));
        }
    }
    // chain adjacent operators with equal parallelism AND a FORWARD exchange (S3.5)
    List<ChainGroup> chains = ChainBuilder.build(vertices, logical);

    Map<String, TaskAssignment> assignments = new HashMap<>();
    int w = 0;
    for (ChainGroup chain : chains) {
        WorkerInfo worker = workers.get(w++ % workers.size());
        assignments.put(chain.id(), new TaskAssignment(worker.workerId(), chain));
    }
    return new ExecutionGraph(logical.jobId(), vertices, assignments);
}
```

8.2 Liveness and the job state machine

Workers open a bidirectional heartbeat stream at registration and send a beat every second. The master declares a worker dead after three missed beats. Because there is nothing to recover to yet, Phase 2's failure response is simply to fail the job - Phase 4 replaces this with restart-from-checkpoint.

```
CREATED -> RUNNING -> FINISHED
      |
      +----> FAILING -> RESTARTING -> RUNNING      (Phase 4)
      |
      +----> FAILED                                (Phase 2 behaviour;
                                                    and when restarts exhausted)
```

Persist every transition to etcd under `/jobs/{jobId}/state` before acting on it, so a restarted master knows what it was in the middle of.

8.3 etcd layout

/jobs/{jobId}/graph	-> serialized JobGraph (written once at submit)
/jobs/{jobId}/state	-> CREATED RUNNING FAILING ...
/jobs/{jobId}/assignments	-> operatorId:subtask -> workerId
/jobs/{jobId}/checkpoints/latest	-> checkpointId + per-task state handle URIs (Phase 4)
/workers/{workerId}	-> host:port, slot count, lease TTL

Workers register under a TTL lease so a crashed worker disappears from etcd on its own. The master watches this prefix rather than polling.

8.4 Done when

- Three worker processes start independently and register with the master.
- A submitted job is distributed across all three; records genuinely cross process boundaries.
- `kill -9` on a worker is detected within ~3 seconds and the job transitions to FAILED with the cause recorded.
- Restarting the master recovers the job graph and assignments from etcd rather than losing them.

9. Phase 3 - Event Time, Watermarks and Session Windows

Phase goal

The engine starts caring about when events happened rather than when they arrived. Adds watermark generation and propagation, keyed state, event-time timers, and the session window operator.

9.1 Watermark generation

A bounded-out-of-orderness generator at the source: track the maximum event time seen, subtract the configured tolerated lateness, emit periodically. Idleness detection is not optional here - it is the bug from Article S6.4, and the engine should demonstrate it rather than hide it.

```
final class BoundedOutOfOrdernessGenerator {
    private final long outOfOrdernessMillis;
    private final long idleTimeoutMillis;
    private long maxEventTime = Long.MIN_VALUE;
    private long lastActivityWallClock = System.currentTimeMillis();
    private boolean idle = false;

    void onEvent(long eventTime) {
        maxEventTime = Math.max(maxEventTime, eventTime);
        lastActivityWallClock = System.currentTimeMillis();
        idle = false;
    }

    /** Called on a fixed interval (default 200ms). */
    Optional<Watermark> onPeriodicEmit() {
        if (System.currentTimeMillis() - lastActivityWallClock > idleTimeoutMillis) {
            idle = true;
            // excluded from downstream min (S6.4)
            return Optional.empty();
        }
        return maxEventTime == Long.MIN_VALUE
            ? Optional.empty()
            : Optional.of(new Watermark(maxEventTime - outOfOrdernessMillis));
    }

    boolean isIdle() { return idle; }
}
```

9.2 Watermark propagation: the minimum rule

Every task tracks one watermark per input channel and takes the minimum as its own event-time clock (Article S6.3). This is also where event-time timers fire.

```
private void handleWatermark(Watermark w, int channelIndex) throws Exception {
    channelWatermarks[channelIndex] = Math.max(channelWatermarks[channelIndex], w.timestamp());

    long min = Long.MAX_VALUE;
    for (int c = 0; c < channelWatermarks.length; c++) {
        if (idleChannels[c]) continue;           // S6.4: skip idle channels
        min = Math.min(min, channelWatermarks[c]);
    }
    if (min == Long.MAX_VALUE || min <= currentWatermark) return;    // no progress

    currentWatermark = min;
    timerService.advanceTo(min, (timestamp, key) -> {                // fires due timers in key order
        stateBackend.setCurrentKey(key);
        ((KeyedOperator) operator).onEventTimer(timestamp, key, collector);
    });
    operator.onWatermark(min, collector);
    collector.emitWatermark(new Watermark(min));                    // broadcast to all outputs
}
```

9.3 The session window operator

Session windows are implemented as merging windows: each event creates a provisional window of [eventTime, eventTime + gap), and overlapping windows for the same key merge. Keeping the aggregation incremental (a running record, not a buffered list) is the S7.3 decision made concrete.

```
public class SessionAggregator implements KeyedOperator<String, ClickEvent, SessionRow> {
    private ValueState<SessionAccumulator> session;    // incremental, not a buffered list
    private ValueState<Long> windowEnd;

    public void processElement(StreamRecord<ClickEvent> rec, Collector<SessionRow> out)
        throws Exception {
        long gap = Duration.ofMinutes(15).toMillis();
        long newEnd = rec.timestamp() + gap;

        SessionAccumulator acc = session.value();
        if (acc == null) acc = SessionAccumulator.empty(rec.value().memberId());
        acc.add(rec.value());                          // count++, searchTerms.add(...)
        session.update(acc);

        Long previousEnd = windowEnd.value();
        if (previousEnd == null || newEnd > previousEnd) {
            windowEnd.update(newEnd);
            ctx.registerEventTimer(newEnd);            // extend the session
        }
    }

    public void onEventTimer(long ts, String memberId, Collector<SessionRow> out)
        throws Exception {
        if (!Objects.equals(windowEnd.value(), ts)) return;    // superseded by a later event
        out.collect(session.value().toRow(ts));
        session.clear();                                       // S9.4: evict, do not leak
        windowEnd.clear();
    }
}
```

The superseded-timer check is the subtle part: extending a session leaves stale timers registered, and firing on one of those would close a session that is still active.

9.4 Done when

- Replaying a fixed file of out-of-order click events produces identical session output on every run - the reproducibility property event time buys (Article S6.1).
- A deliberately silent partition stalls all output until idleness detection is enabled, then does not. This is Article S6.4, demonstrated rather than described.

10. Phase 4 - Checkpointing, Barrier Alignment and Recovery

Phase goal

The centrepiece. Consistent distributed snapshots taken without stopping the stream, and correct recovery from them. Also swaps the in-memory state backend for RocksDB.

10.1 The coordinator

```
public void triggerCheckpoint() {
    long id = checkpointIdCounter.incrementAndGet();
    PendingCheckpoint pending = new PendingCheckpoint(id, allTaskIds(), clock.millis());
    pendingCheckpoints.put(id, pending);

    for (TaskId source : sourceTaskIds()) {           // sources only
        workerClient(source).triggerCheckpoint(id, pending.triggerTimestamp());
    }
    scheduler.schedule(() -> abortIfIncomplete(id), checkpointTimeout, MILLISECONDS);
}

public void onAcknowledge(CheckpointAck ack) {
    PendingCheckpoint pending = pendingCheckpoints.get(ack.checkpointId());
    if (pending == null) return;                       // late ack for an aborted checkpoint
    pending.acknowledge(ack.taskId(), ack.stateHandleUri());

    if (pending.isFullyAcknowledged()) {               // S10.4: every task, not just sinks
        CompletedCheckpoint done = pending.finalize();
        etcd.put("/jobs/" + jobId + "/checkpoints/latest", done.serialize());
        pendingCheckpoints.remove(ack.checkpointId());
        for (TaskId sink : sinkTaskIds()) {           // S11.3: this is the 2PC commit signal
            workerClient(sink).notifyCheckpointComplete(done.id());
        }
    }
}
```

10.2 Barrier alignment

The mechanism the article builds toward. A task with multiple input channels blocks each channel as its barrier arrives, continuing to process the others, until all barriers are in.

```

private void handleBarrier(CheckpointBarrier barrier, int channelIndex) throws Exception {
    if (inputChannels.length == 1) { // single input: no alignment needed
        snapshotAndForward(barrier);
        return;
    }
    if (alignmentStart == 0) alignmentStart = System.nanoTime();

    blockedChannels.set(channelIndex); // stop consuming THIS channel;
                                        // its later records are buffered
    if (blockedChannels.cardinality() < inputChannels.length) {
        return; // keep processing unblocked channels
    }
    // all barriers received -> snapshot, forward, then drain what we buffered
    long alignmentMillis = (System.nanoTime() - alignmentStart) / 1_000_000;
    metrics.alignmentTime.record(alignmentMillis); // S10.5, made observable
    alignmentStart = 0;

    snapshotAndForward(barrier);
    blockedChannels.clear();
    drainBufferedRecords(); // order matters: after forwarding
}

private void snapshotAndForward(CheckpointBarrier barrier) throws Exception {
    StateHandle handle = stateBackend.snapshot(barrier.checkpointId(), checkpointDir);
    collector.emitBarrier(barrier); // downstream first...
    master.acknowledgeCheckpoint(taskId, barrier.checkpointId(), handle.uri()); // ...then ack
}

```

Forwarding the barrier before acknowledging keeps the barrier moving through the graph while the master-side bookkeeping happens - the ack is not on the critical path.

10.3 Source snapshots and recovery

A source task's state is its Kafka offsets. This is the resettable-source requirement from Article S11.1 - without it nothing below works.

```

// source: on barrier, snapshot offsets rather than committing them to Kafka
StateHandle snapshot(long checkpointId) {
    return write(checkpointId, consumer.assignment().stream()
        .collect(toMap(tp -> tp.toString(), consumer::position)));
}

// recovery, driven by the master (Article S10.6 - the WHOLE job restarts):
// 1. cancel every task of the job across all workers
// 2. read /jobs/{jobId}/checkpoints/latest from etcd
// 3. redeploy all tasks, each with its state handle in the deployment message
// 4. each task calls stateBackend.restore(handle); sources seek() to stored offsets
// 5. resume

```

Restarting every task rather than only the failed one is not a simplification we are taking - it is what the baseline algorithm requires, because the rest of the graph has already moved past the checkpoint the failed task is being reset to.

10.4 Done when

- Checkpoints complete on a 10-second interval with alignment time visible per task.
- `kill -9` on a worker mid-session causes a full-job restart from the last completed checkpoint, and final session counts match a clean run exactly.
- Checkpoint duration and state size are reported on the heartbeat and visible in the status API.

11. Phase 5 - The Interval Join

Phase goal

Match a RESULT_CLICK to a BORROW for the same member and catalog item within 30 minutes - buffering both sides, evicting by watermark.

11.1 State layout and eviction

Both sides are buffered, keyed by (memberId, catalogItemId), each entry stamped with its event time. Eviction is driven purely by the watermark - the mechanism from Article S8.3.

```
public class ConversionJoin implements KeyedOperator<String, Either<Click,Borrow>, Conversion> {
    private static final long UPPER_BOUND = Duration.ofMinutes(30).toMillis();
    private static final long LOWER_BOUND = 0;

    private ListState<TimestampedClick> clickBuffer;
    private ListState<TimestampedBorrow> borrowBuffer;

    public void processElement(StreamRecord<Either<Click,Borrow>> rec,
                               Collector<Conversion> out) throws Exception {
        if (rec.value().isLeft()) {
            Click c = rec.value().left();
            clickBuffer.add(new TimestampedClick(c, rec.timestamp()));
            for (TimestampedBorrow b : borrowBuffer.get()) { // match backwards
                if (inInterval(rec.timestamp(), b.ts())) out.collect(Conversion.of(c, b.borrow()));
            }
        } else {
            Borrow b = rec.value().right();
            borrowBuffer.add(new TimestampedBorrow(b, rec.timestamp()));
            for (TimestampedClick c : clickBuffer.get()) { // match forwards
                if (inInterval(c.ts(), rec.timestamp())) out.collect(Conversion.of(c.click(), b));
            }
        }
    }

    private boolean inInterval(long clickTs, long borrowTs) {
        long delta = borrowTs - clickTs;
        return delta >= LOWER_BOUND && delta <= UPPER_BOUND;
    }
}
```

```
/** Eviction: a buffered element can go once no future match is possible for it. */
public void onWatermark(long watermark, Collector<Conversion> out) throws Exception {
    // a click at t can still match a borrow up to t + UPPER_BOUND
    evictIf(clickBuffer, c -> c.ts() + UPPER_BOUND < watermark);
    // a borrow at t can still match a click as early as t - UPPER_BOUND
    evictIf(borrowBuffer, b -> b.ts() < watermark - UPPER_BOUND);
}
```

Note what a missing bound would mean: with no UPPER_BOUND, neither predicate can ever be true, and both buffers grow for the life of the job (Article S8.3).

11.2 Done when

- Clicks and borrows arriving in either order produce exactly one conversion row per match.
- Buffer size, exposed as a metric, rises then plateaus rather than growing without bound.
- Deliberately lagging one input stream visibly inflates the other side's buffer - the slower-stream effect from Article S8.3, observable on a chart.

12. Phase 6 - The Transactional Iceberg Sink

Phase goal

End-to-end exactly-once output: data files written during a checkpoint interval become visible in one atomic Iceberg commit, and only once the checkpoint completes.

12.1 The two-phase mapping

2PC step	In this engine	Iceberg mechanism
Begin transaction	Sink receives CheckpointBarrier for checkpoint N	Open a new set of data files for this interval
Write (uncommitted)	Sink appends rows as they arrive	Parquet data files written to object storage, referenced by no snapshot
Pre-commit / vote	Sink snapshots the list of written file paths into its own state, then acknowledges checkpoint N	Files exist and are durable, still invisible to readers
Commit	Master notifies the sink that checkpoint N completed	AppendFiles commit - one atomic metadata swap makes every file visible at once
Abort / recover	Job restarts from checkpoint N-1	Uncommitted files were never in any snapshot: nothing to roll back, they are simply orphaned and cleaned up later

```
public class IcebergSink implements Operator<SessionRow, Void>, CheckpointListener {
    private final Table table;                // resolved from the Iceberg catalog
    private ListState<String> pendingFiles;    // survives failure with the checkpoint
    private DataWriter<Record> currentWriter;

    public void processElement(StreamRecord<SessionRow> rec, Collector<Void> out)
        throws Exception {
        currentWriter.write(toIcebergRecord(rec.value())); // written, not visible
    }

    /** Called by the runtime as part of snapshotting, before acknowledging. */
    public void preCommit(long checkpointId) throws Exception {
        currentWriter.close();
        for (DataFile f : currentWriter.result().dataFiles()) pendingFiles.add(f.path().toString());
        currentWriter = openNewWriter(); // next interval starts now
    }

    /** Called only once the master confirms ALL tasks acknowledged (S10.4, S11.3). */
    public void notifyCheckpointComplete(long checkpointId) throws Exception {
        AppendFiles append = table.newAppend();
        for (String path : pendingFiles.get()) append.appendFile(dataFileFor(path));
        append.commit(); // atomic snapshot swap
        pendingFiles.clear();
    }
}
```

commit() must tolerate being called twice for the same checkpoint - a crash between commit and state clearing will replay it. Iceberg commits are idempotent per file set, which is what makes this safe (Article S11.2).

12.2 Done when

- A query against the Iceberg table returns nothing for an in-flight checkpoint interval, then the whole interval at once when the checkpoint completes.

- Killing a worker mid-interval leaves orphaned Parquet files that no snapshot references, and the table contains each session row exactly once.
- Shortening the checkpoint interval visibly improves freshness and visibly increases the number of small files - the trade-off from Section 3.2, felt directly.

13. Phase 7 - Status API and the Four Demos

13.1 The status API

```
GET /jobs                -> [ { jobId, name, state, startedAt } ]
GET /jobs/{jobId}        -> { state, vertices[], restartCount }
GET /jobs/{jobId}/tasks  -> [ { operatorId, subtask, worker, state,
                             recordsIn, recordsOut, watermark,
                             backpressured } ]
GET /jobs/{jobId}/checkpoints -> { intervalMs, completed, failed,
                                   history: [ { id, durationMs, stateBytes,
                                                alignmentMs, completedAt } ] }
GET /jobs/{jobId}/lag     -> { perPartition: [...], maxLagMillis }
POST /jobs/{jobId}/cancel
```

Exporting these as Prometheus metrics alongside the JSON endpoints costs little and makes the demos far more legible on a Grafana chart than in a terminal.

13.2 The four demos

Each demo makes an article claim falsifiable rather than merely stated.

#	Demo	Procedure	Proves
1	Worker dies mid-window	Run a fixed replay; kill -9 a worker holding session state halfway; let the job recover and finish. Diff the Iceberg table against a clean run.	Identical output. Exactly-once state (S10.6) plus transactional sink (S11).
2	Master dies	kill -9 the master while workers are processing. Observe workers continue; restart the master and watch it recover graph, assignments and checkpoint pointer from etcd.	Lost coordination is not lost work (S4.5). Recovery needs durable metadata, not in-memory state.
3	Late event	Inject an event whose timestamp precedes the current watermark. Then re-run with allowed lateness configured.	Default drop vs side output vs updated result (S7.4); the sink overwrites rather than duplicating (S11.2).
4	Hot key	Skew 60% of traffic onto one catalogItemId. Watch one subtask backpressure while siblings idle and its checkpoint duration climb. Then enable salted two-phase aggregation and re-run.	Key-based exchange guarantees grouping, not load balance (S13.2). Rebalance would be the wrong fix; salting is the right one.

Demo 4 is the one that changes how people think

It fails while looking entirely healthy: every process up, no errors in any log, aggregate throughput acceptable, one subtask quietly drowning. Run it before you think you need it.

14. Local Environment

```

services:
  kafka:          # KRaft mode, single broker, 4 partitions per topic
  etcd:           # single node, no TLS - metadata store for the master
  minio:          # S3-compatible: checkpoint storage + Iceberg warehouse
  iceberg-rest:   # Iceberg REST catalog pointed at MinIO
  master:         # our JobMaster + StatusApi (ports 8080 REST, 9090 gRPC)
  worker-1:       # our TaskWorker
  worker-2:
  worker-3:
  prometheus:     # scrapes master + workers
  grafana:        # dashboards for the four demo signals
  trino:          # optional: query the Iceberg output table

```

docker compose up starts everything; a seed job publishes a replayable fixture of click and borrow events so every demo is reproducible rather than dependent on live traffic.

Suggested module layout

```

engine-api/      StreamElement, Operator, StateBackend, JobGraph (no runtime deps)
engine-runtime/  Task, run loop, watermark + barrier handling, transport
engine-master/   scheduler, checkpoint coordinator, etcd client, status API
engine-worker/   worker bootstrap, task deployment, gRPC servers
engine-connectors/ Kafka source, Iceberg sink, console sink
lms-job/         the LMS clickstream job written against engine-api only

```

lms-job depending only on engine-api is a useful discipline: it keeps user-facing abstractions honest and makes the API surface obvious.

15. Known Limitations and Extensions

Limitation	Why it is out of scope	Extension if you want it
Master is a single point of failure	Leader election is Part 6 material; the article is explicit that Flink has the same property without HA configured.	etcd already holds the metadata - add an etcd lease-based leader election and a standby master that watches the same keys.
No dynamic rescaling	Parallelism is fixed at submission. Key groups are implemented, so the hard part is already done.	Add a savepoint command, then restore with a different parallelism and let KeyGroupAssigner redistribute.
No savepoints	Checkpoints exist; savepoints are checkpoints with a retention policy and stable operator IDs - which the job graph already assigns.	Add POST /jobs/{id}/savepoint and a --fromSavepoint submit flag (Article S12.2).
Aligned checkpoints only	Alignment is the mechanism being taught; unaligned checkpointing would obscure it.	Persist in-flight buffered records as part of the snapshot and skip channel blocking.
No SQL or higher-level API	Framework DSLs are Part 9B and 9C territory.	A minimal SQL parser producing a JobGraph would be an instructive follow-on project.

Where this lands in the series

Part 9B takes these same mechanisms into Apache Storm, Apache Flink, Spark Structured Streaming and Kafka Streams - four production answers to the problems this engine just made you solve. Reading their architecture documentation after finishing Phase 7 is a noticeably different experience from reading it before.